



Invoice Payment Approval Platform

Technical Documentation & User Guide

An internal platform for submitting invoice payment requests with supporting documents, routing them through a configurable multi-level approval workflow, processing payments, and tracking every action through dashboards, reports, Excel exports and a complete audit trail.

Laravel 13.18

PHP 8.3

Vue 3

Vuetify

Pinia

Chart.js 4

MySQL 8.4

Vite 8

Laravel Excel

Contents

1 Introduction

1.1 Purpose · 1.2 Feature summary · 1.3 Technology stack

2 System Architecture

2.1 High-level design · 2.2 Authentication & sessions · 2.3 Project layout

3 Roles & Permissions

3.1 The four roles · 3.2 Permission matrix · 3.3 Data visibility rules

4 Functional Guide

4.1 Submitting a request · 4.2 Supporting documents · 4.3 Approval workflow · 4.4 Payment processing · 4.5 Status lifecycle · 4.6 Dashboard · 4.7 Reports & Excel export · 4.8 Audit trail · 4.9 Administration

5 Data Model

5.1 Entity overview · 5.2 Table specifications · 5.3 Referential integrity

6 API Reference

6.1 Conventions · 6.2 Route inventory · 6.3 Endpoint details · 6.4 Validation rules

7 Frontend Application

7.1 SPA routes & guards · 7.2 Pages · 7.3 State management · 7.4 HTTP & CSRF wiring · 7.5 Theme & visual language

8 Configuration

8.1 Business configuration (config/paf.php) · 8.2 Environment (.env)

9 Installation & Operations

9.1 Requirements · 9.2 First-time setup · 9.3 Daily development · 9.4 Production checklist

10 Seeded Accounts & Demo Data

11 Security Notes & Known Limitations

A Appendix — Audit action catalog · Status & priority reference

1 Introduction

What PAF is, what it does, and what it is built with.

1.1 Purpose

PAF (Payment Approval Flow) is an internal web platform that standardises how invoice payments are requested, approved and executed. Employees submit a payment request against a vendor invoice, attach the supporting documents (invoice copy, purchase order, delivery note...), and the platform routes the request through an **amount-based, multi-level approval chain**. Once fully approved, the finance team schedules and executes the payment, closing the loop with a payment reference. Every step — from creation to payment — is recorded in an immutable audit trail, and the whole portfolio is observable through dashboards, filterable reports and Excel exports.

1.2 Feature summary

Capability	What it provides
Form submission	Structured payment-request form: vendor details, invoice number & dates, amount + VAT with live total, category, department, cost center, payment method, priority and justification. Save as draft or submit immediately.
File uploads	Up to 10 supporting documents per request, 10 MB each (PDF, images, Office documents, CSV, TXT). Stored on the private local disk; downloads are permission-checked.
Multi-level approval workflow	Configurable approval levels with amount thresholds (seeded: L1 Department Manager — always; L2 Finance Director — $\geq 10,000$; L3 CFO — $\geq 50,000$). Requests pass every applicable level in order; rejection at any level stops the chain with a mandatory reason; rejected requests can be edited and resubmitted for a fresh cycle.
Payment lifecycle tracking	Seven statuses (draft → pending approval → approved → scheduled → paid, plus rejected / cancelled) with timestamps for each milestone and a payment reference recorded at execution.
Dashboard analytics	KPI cards (totals, pending, awaiting payment, paid this month), status distribution donut, 6-month submitted-vs-paid trend, top-5 vendors, spend by category, recent requests, and a personal approval-queue alert for approvers.
Reporting & Excel export	Report builder with status/department/category/vendor/date filters, per-status summary cards, and a 23-column <code>.xlsx</code> export (Laravel Excel / PhpSpreadsheet) honouring the same filters and the caller's data visibility.
Audit trail	Every action (create, update, submit, approve, reject, schedule, pay, cancel, document upload/delete, login/logout, exports, admin changes) is logged with actor, timestamp, IP address and — for updates — old/new values. Per-invoice timeline plus an admin-wide searchable log.
Administration	User management (roles, approver levels, activation) and approval-level configuration (thresholds, names, active flags) — all from the UI, no code changes needed.

1.3 Technology stack

Layer	Technology	Version constraint	Notes
Backend framework	Laravel	<code>^13.8</code> (13.18.0 installed)	Slim Laravel 11+ bootstrap style (<code>bootstrap/app.php</code>)
Language	PHP	<code>^8.3</code>	Runs on PHP 8.3.30 locally
Database	MySQL	8.4.x	Database <code>gallega_paf_db</code> , utf8mb4
Excel export	maatwebsite/excel	<code>^3.1</code>	PhpSpreadsheet under the hood; requires <code>ext-zip</code> , <code>ext-gd</code>
Frontend framework	Vue	<code>^3.5</code>	Composition API (<code><script setup></code>), SPA
UI library	Vuetify	<code>^4.1</code>	Material Design components, light theme
State	Pinia	<code>^3.0</code>	<code>auth</code> , <code>meta</code> , <code>notify</code> stores
Routing	vue-router	<code>^5.1</code>	HTML5 history mode + role guards
Charts	Chart.js	<code>^4.5</code>	Doughnut + bar charts on the dashboard
HTTP client	axios	<code>^1.18</code>	Session cookies + CSRF, <code>/api</code> base
Build tool	Vite	<code>^8.0</code>	<code>laravel-vite-plugin</code> , <code>vite-plugin-vuetify</code> (auto-import)
Icons	@mdi/font	<code>^7.4</code>	Material Design Icons webfont

`tailwindcss` and `@tailwindcss/vite` remain in `package.json` from the Laravel skeleton but are *not* used — the Vite config compiles only `resources/js/main.js` with the Vue and Vuetify plugins.

2 System Architecture

A single Laravel application serving a Vue SPA and a session-authenticated JSON API.

2.1 High-level design

PAF is deliberately a **single deployable**: one Laravel application serves both the compiled Vue single-page application and the JSON API it consumes. There is no separate API host, no token infrastructure and no CORS surface.



Figure 1 — Deployment shape: one Laravel app serves the SPA shell, the JSON API, the database and private file storage.

- **SPA catch-all:** `Route::view('/{any?}', 'app')->where('any', '^(?!api).*$')` returns `resources/views/app.blade.php` for every non- `/api` path. The Blade shell carries the CSRF meta tag and the Vite-built bundle; vue-router takes over client-side.
- **API prefix inside web.php:** all 31 API routes live under `Route::prefix('api')` in `routes/web.php` — there is no `routes/api.php`. This keeps them inside the `web` middleware group: database-backed sessions, cookies and CSRF protection apply to every call.
- **JSON errors everywhere:** `bootstrap/app.php` registers `$exceptions->shouldRenderJsonWhen(fn ($request) => $request->is('api/*'))`, so validation failures, auth failures and 404s on `/api/*` always render as JSON — the SPA never receives an HTML error page or redirect.
- **Role gate:** a single custom middleware, `EnsureRole` (alias `role`), guards the finance (`role:finance,admin`) and administration (`role:admin`) route groups with a strict role whitelist; everything else authorises inside controllers/services.
- **Business logic placement:** the approval state machine is isolated in `App\Services\ApprovalService` (submit / approve / reject, all transactional), and audit writing in `App\Services\AuditLogger`. Controllers stay thin: validation, authorisation, orchestration.

2.2 Authentication & sessions

- Classic Laravel **session authentication** (`web` guard) — no Sanctum tokens, no JWT. The SPA is same-origin, so the session cookie authenticates every API call transparently.
- `POST /api/login` validates credentials with `Auth::attempt()` (optional `remember` flag), rejects deactivated accounts (`is_active = false`) even with a correct password, regenerates the session ID against fixation, and writes a `login` audit entry. The route is throttled at **10 attempts per minute**.
- `POST /api/logout` logs the audit entry, logs out the `web` guard, invalidates the session and regenerates the CSRF token.
- **CSRF:** Laravel issues the `XSRF-TOKEN` cookie, which axios echoes automatically as `X-XSRF-TOKEN`; the Blade shell's `<meta name="csrf-token">` is wired as a fallback `X-CSRF-TOKEN` header for the first request after a full page load.

- Sessions, cache and queues are all **database-backed** (`SESSION_DRIVER=database` , `CACHE_STORE=database` , `QUEUE_CONNECTION=database`) — no Redis dependency.

2.3 Project layout

```
paf/
├─ app/
│   ├─ Exports/InvoicesExport.php           # 23-column Excel export definition
│   ├─ Http/
│   │   ├─ Controllers/                     # Auth, Invoice, Document, Approval, Payment,
│   │   │   │                               # Dashboard, Report, AuditLog, User, ApprovalLevel, Meta
│   │   │   └─ Middleware/EnsureRole.php    # 'role:...' middleware alias
│   │   └─ Models/                          # User, Invoice, InvoiceDocument, InvoiceApproval,
│   │       │                               # ApprovalLevel, AuditLog
│   │       └─ Services/
│   │           └─ ApprovalService.php       # submit / approve / reject state machine
│   │           └─ AuditLogger.php           # single entry point for audit writes
├─ bootstrap/app.php                        # routing, role alias, JSON exception rendering
├─ config/paf.php                           # business lists: categories, departments, currencies,
│                                           # payment methods, priorities, upload limits
├─ database/
│   ├─ migrations/                          # users(+PAF fields), approval_levels, invoices,
│   │   │                               # invoice_documents, invoice_approvals, audit_logs
│   │   └─ seeders/                       # ApprovalLevelSeeder, UserSeeder, DemoDataSeeder
├─ resources/
│   ├─ js/                                  # Vue SPA (see §7)
│   └─ views/app.blade.php                 # SPA shell (CSRF meta + @vite)
├─ routes/web.php                           # /api/* endpoints + SPA catch-all
└─ storage/app/private/invoices/{id}/      # uploaded documents (private disk)
```

3 Roles & Permissions

Four roles cover the whole payment flow; approvers additionally carry a numeric level.

3.1 The four roles

Role	Who	What they do
requester	Any employee raising payments	Creates, edits and submits their own requests; cancels while draft/pending; tracks progress; sees only their own data.
approver	Managers/directors/CFO, each with an approval_level (1–10)	Reviews the queue routed to their level; approves (optional comment) or rejects (mandatory reason). Also sees requests that carry an approval row at their level plus anything they submitted themselves.
finance	Accounts-payable team	Processes fully-approved requests: schedules payment dates and records execution with a payment reference. Sees all requests.
admin	System administrators	Everything: acts at any approval level, manages users and approval levels, reads the global audit trail. Sees all requests.

3.2 Permission matrix

Action	Requester	Approver	Finance	Admin
Create / edit / submit own requests	✓	✓	✓	✓ (+ others' editable requests — draft/rejected)
Cancel own draft / pending request	✓	✓	✓	✓ (any)
Delete own draft	✓	✓	✓	✓ (any)
View all requests	✗ own only	● own + routed to their level	✓	✓
Approve / reject at current level	✗	● only when current_level = their level	✗	✓ any level
Approval queue (/approvals)	✗	✓ their level	✗	✓ all levels
Schedule payment / mark paid	✗	✗	✓	✓
Reports & Excel export	● own data	● visible data	✓ all data	✓ all data
Global audit trail	✗	✗	✗	✓
Manage users & approval levels	✗	✗	✗	✓

✓ full · ● conditional · ✗ none. Download of a request's documents follows the same visibility rule as viewing the request.

3.3 Data visibility rules

Visibility is enforced in one Eloquent scope, `Invoice::visibleTo($user)`, applied to lists, dashboards and reports alike:

1. **Admin & finance** (`canViewAllInvoices()`) — unrestricted.
2. **Approver** — requests they submitted, *plus* any request having an approval row at their level, whatever that row's status — including requests they already acted on. Because resubmission rebuilds the chain rows (§4.3), this visibility lasts only until the request is resubmitted with a chain that no longer includes their level.
3. **Requester** (and any other role) — only requests where `submitted_by` is the user.

The same three-way rule is re-implemented for single-record access in `InvoiceController::authorizeView()` and `DocumentController::authorizeAccess()` — see §11 for the maintenance note.

4 Functional Guide

How each capability behaves, module by module.

4.1 Submitting a payment request

Payment Requests → **New Request** opens the submission form. Every request receives a unique reference `PAF-YYYY-NNNNN` (year-scoped, sequential, zero-padded to five digits — e.g. `PAF-2026-00061`). The requester can *Save as Draft* (no workflow yet) or *Submit for Approval* (drafting and submitting in one step).

Field	Required	Rules & behaviour
Vendor name	yes	Free text, max 255
Vendor email	no	Valid email, max 255
Vendor TRN	no	Tax registration number, max 50
Invoice number	yes	Vendor's invoice number, max 100 (not required to be unique — the platform's own <code>reference_no</code> is the unique key)
Invoice date	yes	Date
Due date	no	Date, must be on/after the invoice date
Currency	yes	One of <code>AED</code> (default), <code>USD</code> , <code>EUR</code> , <code>GBP</code> , <code>SAR</code>
Amount	yes	Numeric, ≥ 0.01
Tax / VAT	no	Numeric ≥ 0 , defaults to 0. Total = round(amount + tax, 2) is computed server-side and shown live in the form
Category	yes	One of the 11 configured categories (§8.1)
Department	yes	One of the 9 configured departments (§8.1)
Cost center	no	Free text, max 100
Payment method	yes	Bank Transfer (default) · Cheque · Cash · Corporate Card
Priority	yes	<code>low</code> · <code>normal</code> (default) · <code>high</code> · <code>urgent</code> — drives queue ordering
Description	no	Justification, max 5,000 characters
Documents	no	See §4.2

Drafts (and rejected requests) remain fully editable by their owner; editing records an `updated` audit entry with the old and new values. Only drafts can be deleted; drafts and pending requests can be cancelled.

4.2 Supporting documents

- **Limits:** up to **10 files** per request, **10 MB** (10,240 KB) each.
- **Accepted types:** `pdf`, `jpg`, `jpeg`, `png`, `webp`, `doc`, `docx`, `xls`, `xlsx`, `csv`, `txt` (content-sniffed by Laravel's `mimes` rule, not just the extension).
- **Storage:** private local disk under `invoices/{invoice_id}/` with a random hashed filename; the original filename is preserved in the database and restored on download.
- **Access control:** downloading goes through `GET /api/documents/{id}/download` which applies the request-visibility rule (§3.3) before streaming the file with its original name.

- **Removal:** only the uploader (or an admin) may delete a document, and only while the request is editable (draft/rejected). Uploads and deletions are audit-logged; downloads are not.
- Documents can be added when creating, and more can be appended when editing an editable request.

4.3 Approval workflow

Routing by amount

Approval levels live in the `approval_levels` table and are managed from the admin UI. A request must pass **every active level whose `min_amount` $\leq$ the request's total**, in ascending level order (`ApprovalLevel::requiredFor()`). With the seeded configuration:

Total (AED)	Chain
< 10,000	L1 Department Manager
10,000 – 49,999.99	L1 Department Manager → L2 Finance Director
$\geq$ 50,000	L1 Department Manager → L2 Finance Director → L3 CFO

Thresholds are inclusive: a request of exactly 10,000 requires L2. Levels are matched purely on amount — there is no category- or department-based routing.

Chain mechanics

1. **Submit** (drafts and rejected requests only): one `invoice_approvals` row is created per required level (snapshotting the level's name), the request becomes `pending_approval` with `current_level` at the lowest level, `submitted_at` is stamped, and any previous approval/rejection timestamps are cleared. *Resubmission starts a fresh cycle*: all prior approval rows are deleted first (the audit log keeps the history).
2. **Who may act:** an approver whose `approval_level` equals the request's `current_level`, or any admin (at any level). Everyone else receives 403; acting on a non-pending request returns a validation error. Levels have no named assignee — any qualified approver at the level may act, first action wins.
3. **Approve:** the current level's row is stamped (approver, optional comment, timestamp). If a higher pending level remains, `current_level` advances to it; otherwise the request becomes `approved` with `approved_at` set and enters the finance queue.
4. **Reject:** requires a reason (max 2,000 chars). The current level's row is marked rejected, the request becomes `rejected` with the reason stored on the request itself. The owner may edit and resubmit — restarting the whole chain.
5. All three operations run inside database transactions and write audit entries.

Approval queue

Approvals (approvers/admins) lists requests pending at the caller's level (admins: all levels), ordered *urgent* → *high* → *normal* → *low*, then oldest submission first. Approve/reject can be done inline from the queue or from the request detail page, which shows the full approval chain timeline with per-level status, actor, timestamp and comments.

4.4 Payment processing

The **Payments** workspace (finance/admin) has three tabs backed by `GET /api/payments/queue` :

- **To Schedule** — fully-approved requests, priority-ordered then oldest-approved first. Actions: *Schedule* (choose a payment date, today or later) or *Mark Paid* directly.

- **Scheduled** — requests with a planned `scheduled_date` . Action: *Mark Paid*.
- **Paid** — executed payments, most recent first, showing the payment reference.

Mark Paid requires a **payment reference** (transaction number, max 100 chars); it stamps `paid_at` (defaults to now, may be supplied) and records the finance user in `paid_by` . Both actions are audit-logged.

4.5 Status lifecycle

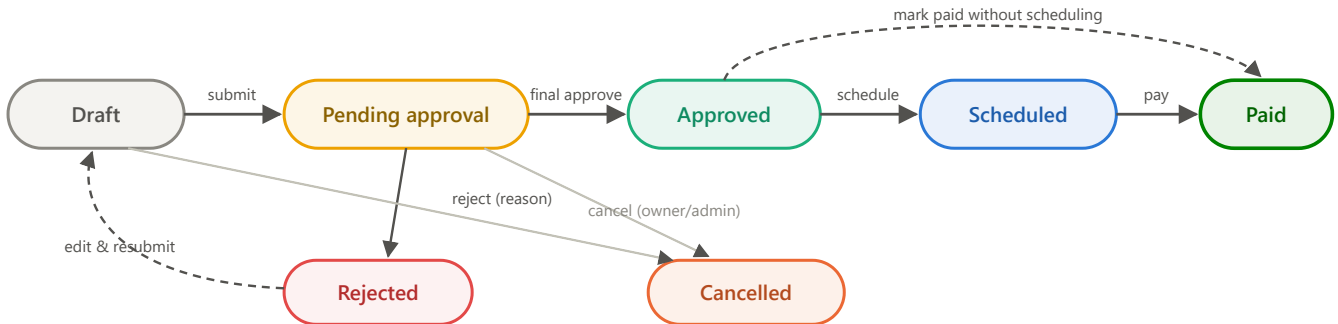


Figure 2 — Payment lifecycle. Rejected requests are editable and can re-enter the chain; cancelled is terminal.

Status	Meaning	Set when	Editable	Exits
draft	Captured but not in workflow	Created (default)	yes	submit · cancel · delete
pending_approval	Awaiting the approver at <code>current_level</code>	Submitted / resubmitted	no	approve (advance/final) · reject · cancel
approved	Chain complete, awaiting finance	Final level approves (<code>approved_at</code>)	no	schedule · mark paid
scheduled	Payment date planned	Finance schedules (<code>scheduled_date</code>)	no	mark paid
paid	Executed — terminal	Finance records reference (<code>paid_at</code> , <code>paid_by</code>)	no	—
rejected	Declined with reason	Any level rejects (<code>rejected_at</code> , <code>rejection_reason</code>)	yes	edit · resubmit
cancelled	Withdrawn — terminal	Owner/admin cancels a draft or pending request	no	—

4.6 Dashboard

The landing page adapts to the signed-in user's visibility (§3.3) and refreshes on every visit. It contains:

- **Approval alert** (approvers/admins): "*N requests are waiting for your approval*" with a shortcut to the queue.
- **KPI cards**: Total Requests (all time), Pending Approval (count + amount), Awaiting Payment (approved + scheduled, count + amount), Paid This Month (count + amount).
- **Requests by Status** — donut of the status distribution, with counts in the legend. Status colors are stable across the entire app (§7.5).

- **Monthly Flow** — grouped bars for the last 6 calendar months (including the current one): total submitted vs total paid, in AED.
- **Top Vendors by Spend** — top 5 vendors by amount (drafts, cancelled and rejected excluded).
- **Spend by Category** — same basis, grouped by category.
- **Recent Requests** — the 8 newest visible requests with quick links.

4.7 Reports & Excel export

Reports combines filters (status multi-select, department, category, vendor contains-match, invoice-date range), live summary cards (matching count and amount, plus one card per status in the result), and a paginated result table. All data respects the caller's visibility — a requester exports only their own requests, finance/admin export everything.

Export to Excel downloads `paf-invoices-YYYYMMDD-HHMMSS.xlsx` with the same filters applied, auto-sized columns and a bold header row. The 23 columns, in order:

Reference No	Vendor	Invoice No	Invoice Date	Due Date	Currency	Amount	Tax	Total	Category
Department	Cost Center	Payment Method	Priority	Status	Requested By	Submitted At	Approved At		
Scheduled Date	Paid At	Payment Reference	Paid By	Rejection Reason					

Amount, Tax and Total are numeric cells; dates are `Y-m-d`, timestamps `Y-m-d H:i`; the payment method exports its display label. Every export writes a `report_exported` audit entry.

4.8 Audit trail

Every state-changing action is written to `audit_logs` through a single service (`AuditLogger::log()`), capturing the acting user, the affected request, an action keyword, a human-readable description, the client IP, a timestamp — and, for edits, JSON snapshots of old and new values.

- **Per-request timeline** — the request detail page shows its full history chronologically with actor and action icons.
- **Global audit trail** (admin) — searchable by text or reference number, filterable by action type (dropdown is populated from the distinct actions actually present), user and date range; 25 entries per page, newest first.

The full catalog of action keywords is in Appendix A.

4.9 Administration

Users

- Create and edit accounts: name, email (unique), password (min 8; on edit, leave blank to keep), role, department, job title.
- Choosing the **approver** role requires an approval level (1–10, picked from the configured levels); the level is cleared automatically when switching to another role.
- Accounts are never deleted — they are **deactivated** (`is_active`), which blocks login with a clear message while preserving history. User changes are audit-logged with before/after snapshots.

Approval levels

- Levels are rows of *level number (1–10, unique) · name · minimum amount · active flag*. A minimum of 0 means "reviews every request".

- Deactivating or deleting a level affects *future submissions only* — requests already in flight keep the chain they were submitted with (each request snapshots its levels), and past history keeps its records.
- All changes are audit-logged (`approval_level_created/updated/deleted`).

5 Data Model

Six domain tables on top of Laravel's framework tables (sessions, cache, jobs).

5.1 Entity overview

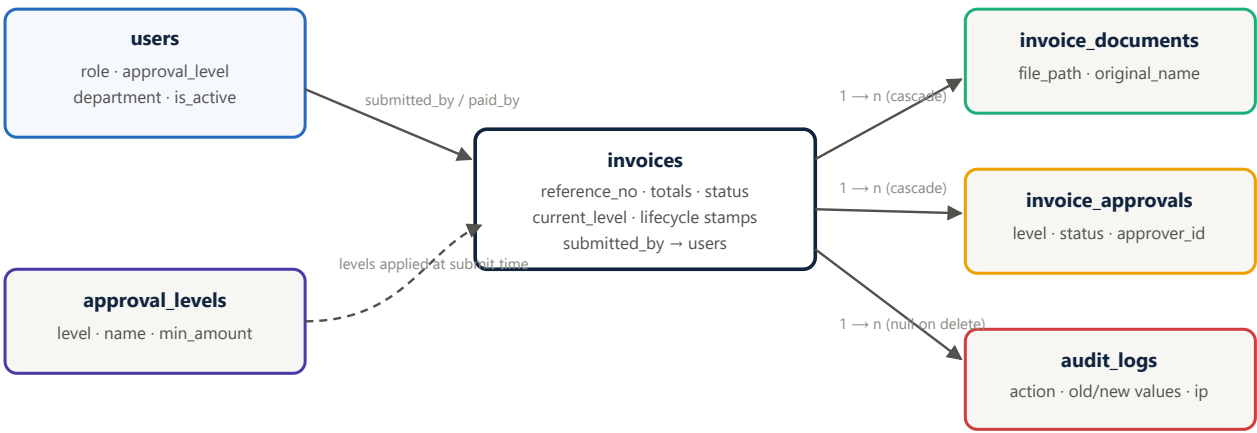


Figure 3 — Domain entities. Approval rows and documents belong to an invoice; audit rows reference it loosely (kept on delete).

5.2 Table specifications

invoices

Column	Type	Null / default	Notes
id	bigint unsigned, AI	—	PK
reference_no	varchar(255)	—	Unique. Format PAF-YYYY-NNNNN , sequence resets each year
vendor_name	varchar(255)	—	Indexed
vendor_email / vendor_trn	varchar(255)	null	TRN = tax registration number
invoice_no	varchar(255)	—	Vendor's number; not unique by design
invoice_date / due_date	date	— / null	
currency	varchar(3)	'AED'	
amount / tax_amount / total_amount	decimal(15,2)	— / 0 / —	total_amount is server-computed and drives level routing
category / department / cost_center	varchar(255)	— / — / null	department indexed
payment_method	varchar(255)	'bank_transfer'	bank_transfer cheque cash card
priority	varchar(255)	'normal'	low normal high urgent
description	text	null	

Column	Type	Null / default	Notes
status	varchar(255)	'draft'	7 values, §4.5. Composite index (status, current_level)
current_level	tinyint unsigned	null	Level currently pending; null outside the chain
submitted_by	FK → users	—	Restrict on delete
submitted_at / approved_at / rejected_at / paid_at	timestamp	null	Lifecycle milestones
rejection_reason	text	null	Copied from the rejecting approver's comment
scheduled_date	date	null	
payment_reference	varchar(255)	null	
paid_by	FK → users	null	Restrict on delete

invoice_approvals

Column	Type	Null / default	Notes
invoice_id	FK → invoices	—	Cascade on delete ; unique with level — one row per level per request
level	tinyint unsigned	—	
level_name	varchar(255)	—	Snapshot of the level's name at submission time
status	varchar(255)	'pending'	pending approved rejected
approver_id	FK → users	null	Filled when acted upon
comments	text	null	Optional on approve, required on reject
acted_at	timestamp	null	

invoice_documents

Column	Type	Null / default	Notes
invoice_id	FK → invoices	—	Cascade on delete
uploaded_by	FK → users	—	
original_name / file_path	varchar(255)	—	Original name shown/download; hashed path on the private disk
mime_type	varchar(255)	null	
size	bigint unsigned	0	Bytes

approval_levels

Column	Type	Null / default	Notes
level	tinyint unsigned	—	Unique , 1–10 enforced by validation
name	varchar(255)	—	e.g. "Finance Director"
min_amount	decimal(15,2)	0	Level applies when request total ≥ this value
is_active	boolean	true	Inactive levels are skipped for new submissions

audit_logs

Column	Type	Null / default	Notes
user_id	FK → users	null	Acting user (null for system)
invoice_id	FK → invoices	null	Set null on invoice delete — trail survives
action	varchar(255)	—	Indexed keyword (Appendix A)
description	varchar(255)	—	Human-readable summary
old_values / new_values	json	null	Populated for updates
ip_address	varchar(45)	null	IPv4/IPv6
created_at	timestamp	CURRENT_TIMESTAMP	No updated_at — rows are write-once. Composite index (invoice_id, created_at)

users (PAF extensions)

Column	Type	Null / default	Notes
role	varchar(255)	'requester'	admin requester approver finance
approval_level	tinyint unsigned	null	Approvers only; matched against invoices.current_level
department / job_title	varchar(255)	null	
is_active	boolean	true	Deactivated users cannot sign in

5.3 Referential integrity

Relationship	On delete	Rationale
invoice_documents.invoice_id → invoices	cascade	Documents have no life outside their request (files are removed via the API path)
invoice_approvals.invoice_id → invoices	cascade	Chain rows are per-request state

Relationship	On delete	Rationale
<code>audit_logs.invoice_id</code> → invoices	set null	The trail must survive request deletion
<code>invoices.submitted_by</code> / <code>paid_by</code> , <code>invoice_approvals.approver_id</code> , <code>invoice_documents.uploaded_by</code> , <code>audit_logs.user_id</code> → users	restrict	Users are deactivated, never deleted — history stays attributable

`users.approval_level` ↔ `approval_levels.level` and `invoices.current_level` ↔ `invoice_approvals.level` are correlated integers without database foreign keys; integrity is enforced by the application (validation + workflow service).

6 API Reference

31 session-authenticated JSON endpoints under `/api`, plus the SPA catch-all.

6.1 Conventions

- **Base URL:** same origin, prefix `/api`. All requests need the session cookie; state-changing requests need the CSRF header (handled automatically by the SPA's axios instance).
- **Errors:** always JSON on `/api/*` — `401 {"message":"Unauthenticated."}`, `403` with a message, `404`, `422 {"message": ..., "errors": {field: [msg]}}` for validation.
- **Pagination:** list endpoints return the standard Laravel paginator (`data`, `total`, `current_page`, `per_page`, `last_page`, ...); page size via `per_page`.
- **Invoice updates use POST** (`POST /api/invoices/{id}`) rather than PUT so multipart file uploads work.

6.2 Route inventory

Method	Path	Handler	Access
POST	<code>/api/login</code>	<code>AuthController@login</code>	guest · throttled 10/min
POST	<code>/api/logout</code>	<code>AuthController@logout</code>	authenticated
GET	<code>/api/me</code>	<code>AuthController@me</code>	authenticated
GET	<code>/api/meta</code>	<code>MetaController@index</code>	authenticated
GET	<code>/api/dashboard</code>	<code>DashboardController@index</code>	authenticated (scoped)
GET	<code>/api/invoices</code>	<code>InvoiceController@index</code>	authenticated (scoped)
POST	<code>/api/invoices</code>	<code>InvoiceController@store</code>	authenticated
GET	<code>/api/invoices/{id}</code>	<code>InvoiceController@show</code>	visibility rule
POST	<code>/api/invoices/{id}</code>	<code>InvoiceController@update</code>	owner/admin · editable status
DELETE	<code>/api/invoices/{id}</code>	<code>InvoiceController@destroy</code>	owner/admin · draft only
POST	<code>/api/invoices/{id}/submit</code>	<code>InvoiceController@submit</code>	owner/admin · draft/rejected
POST	<code>/api/invoices/{id}/cancel</code>	<code>InvoiceController@cancel</code>	owner/admin · draft/pending
GET	<code>/api/documents/{id}/download</code>	<code>DocumentController@download</code>	visibility rule
DELETE	<code>/api/documents/{id}</code>	<code>DocumentController@destroy</code>	uploader/admin · editable request
GET	<code>/api/approvals/pending</code>	<code>ApprovalController@pending</code>	approver (their level) / admin (all)
POST	<code>/api/invoices/{id}/approve</code>	<code>ApprovalController@approve</code>	level approver / admin
POST	<code>/api/invoices/{id}/reject</code>	<code>ApprovalController@reject</code>	level approver / admin
GET	<code>/api/payments/queue</code>	<code>PaymentController@queue</code>	<code>role:finance,admin</code>
POST	<code>/api/invoices/{id}/schedule</code>	<code>PaymentController@schedule</code>	<code>role:finance,admin</code> · approved
POST	<code>/api/invoices/{id}/mark-paid</code>	<code>PaymentController@markPaid</code>	<code>role:finance,admin</code> · approved/scheduled
GET	<code>/api/reports</code>	<code>ReportController@index</code>	authenticated (scoped)

Method	Path	Handler	Access
GET	/api/reports/export	ReportController@export	authenticated (scoped)
GET	/api/audit-logs	AuditLogController@index	role:admin
GET	/api/audit-logs/actions	AuditLogController@actions	role:admin
GET	/api/users	UserController@index	role:admin
POST	/api/users	UserController@store	role:admin
PUT	/api/users/{id}	UserController@update	role:admin
GET	/api/approval-levels	ApprovalLevelController@index	role:admin
POST	/api/approval-levels	ApprovalLevelController@store	role:admin
PUT	/api/approval-levels/{id}	ApprovalLevelController@update	role:admin
DELETE	/api/approval-levels/{id}	ApprovalLevelController@destroy	role:admin
GET	/ {any} (non-api)	SPA shell (app.blade.php)	public shell; data requires login

The framework additionally exposes `GET /up`, a public health-check endpoint registered in `bootstrap/app.php` (returns 200 when the application boots).

6.3 Endpoint details

Invoices

GET /api/invoices

Query params: `page`, `per_page` (default 15), `mine` (bool), `status` (comma list or array), `department`, `date_from` / `date_to` (on invoice date), `q` (matches reference, vendor, invoice #), `sort` (`created_at` | `invoice_date` | `due_date` | `total_amount` | `status`, default `created_at`), `dir` (`asc` / `desc`). Rows include `submitter {id, name, department}`.

POST /api/invoices • **POST** /api/invoices/{id}

Multipart body with the fields of §6.4 plus `documents[]` files and `action = draft | submit`. Create returns **201** with the invoice (+documents +approvals); update requires owner/admin and draft/rejected status. With `action=submit` the approval chain is built immediately after saving. Audit: `created` / `updated` (+ `document_uploaded` per file, + `submitted`).

GET /api/invoices/{id}

Full detail: submitter, payer, documents (+uploader), approval chain (+approver names), and the complete audit timeline (+actors). 403 outside the visibility rule.

POST /api/invoices/{id}/submit • /cancel • **DELETE** /api/invoices/{id}

Owner or admin. Submit: draft/rejected → pending (422 otherwise; 422 if no active levels are configured). Cancel: draft/pending → cancelled. Delete: drafts only. Audit: `submitted` / `cancelled` / `deleted`.

Approvals

GET /api/approvals/pending

Pending requests at the caller's level (admin: every level), ordered urgent→high→normal→low then oldest submitted. 403 for non-approvers.

POST /api/invoices/{id}/approve · /reject

Body: `comments` — optional ≤2000 on approve, **required** on reject. Allowed for admins and for approvers whose level equals the request's `current_level`; otherwise 403. Non-pending requests → 422. Approve advances the chain or finalises; reject terminates with the reason. Audit: `approved` / `rejected`.

Payments (finance/admin)

GET /api/payments/queue?status=approved|scheduled|paid

Default `approved`; other values → 422. Priority-ordered; paid tab newest first.

POST /api/invoices/{id}/schedule · /mark-paid

Schedule: `scheduled_date` required, today or later; only from `approved`. Mark paid: `payment_reference` required ≤100, optional `paid_at`; from `approved` or `scheduled`. Audit: `scheduled` / `paid`.

Dashboard, reports, meta

GET /api/dashboard

Returns `cards` (total, pending, awaiting_payment, paid_this_month — counts and amounts), `my_queue`, `status_distribution`, `monthly` (6 entries: month label, submitted, paid), `top_vendors` (≤5), `by_category`, `recent` (≤8). All scoped by visibility except `my_queue`, which counts the approver's actual desk.

GET /api/reports · **GET** /api/reports/export

Filters: `status` (multi), `department`, `category`, `vendor` (contains), `date_from` / `date_to`. Index returns `summary` (per-status count/amount), `totals`, and paginated `rows` (default 25). Export streams the filtered `.xlsx` (\$4.7) and logs `report_exported`.

GET /api/meta

Dropdown data for the SPA: categories, departments, currencies, payment methods, priorities, statuses, roles, all approval levels (including inactive), and upload limits.

Administration (admin)

GET/POST /api/users · **PUT** /api/users/{id}

List (search `q`, filter `role`), create (201), update. No delete — deactivate via `is_active`. Audit: `user_created` / `user_updated` (with role/level/department/active before-after snapshots).

GET/POST /api/approval-levels · **PUT/DELETE** /api/approval-levels/{id}

CRUD for the level configuration. Audit: `approval_level_created/updated/deleted`.

[GET /api/audit-logs](#) · [GET /api/audit-logs/actions](#)

Filters: `q` (description or invoice reference), `action`, `user_id`, `date_from` / `date_to`, `per_page` (default 25), newest first.
`/actions` returns the distinct action keywords for the filter dropdown.

6.4 Invoice validation rules

Field	Rules (identical on create and update)
<code>vendor_name</code>	required · string · max 255
<code>vendor_email</code>	nullable · email · max 255
<code>vendor_trn</code>	nullable · string · max 50
<code>invoice_no</code>	required · string · max 100
<code>invoice_date</code>	required · date
<code>due_date</code>	nullable · date · after_or_equal:invoice_date
<code>currency</code>	required · in configured currencies
<code>amount</code>	required · numeric · min 0.01 · max 999,999,999,999
<code>tax_amount</code>	nullable · numeric · min 0 · max 999,999,999,999 (defaults 0; total computed server-side)
<code>category</code> / <code>department</code>	required · in configured lists
<code>cost_center</code>	nullable · string · max 100
<code>payment_method</code>	required · in <code>bank_transfer cheque cash card</code>
<code>priority</code>	required · in <code>low normal high urgent</code>
<code>description</code>	nullable · string · max 5,000
<code>documents</code> / <code>documents.*</code>	array max 10 · each: file · mimes <code>pdf,jpg,jpeg,png,webp,doc,docx,xls,xlsx,csv,txt</code> · max 10,240 KB

7 Frontend Application

A Vue 3 + Vuetify SPA under `resources/js`, built by Vite into `public/build`.

7.1 SPA routes & guards

Path	Page component	Access
<code>/login</code>	LoginPage	guests (logged-in users are bounced to the dashboard)
<code>/dashboard</code>	DashboardPage	all authenticated
<code>/invoices</code>	invoices/InvoiceListPage	all authenticated
<code>/invoices/new</code> <code>/invoices/:id/edit</code>	invoices/InvoiceFormPage	all authenticated (edit re-checked server-side)
<code>/invoices/:id</code>	invoices/InvoiceDetailPage	visibility rule (server-enforced)
<code>/approvals</code>	ApprovalsPage	approver · admin
<code>/payments</code>	PaymentsPage	finance · admin
<code>/reports</code>	ReportsPage	all authenticated
<code>/audit-log</code>	AuditLogPage	admin
<code>/admin/users</code> <code>/admin/approval-levels</code>	admin/UsersPage admin/ApprovalLevelsPage	admin

A global `beforeEach` guard loads the session user once (`GET /api/me`), redirects guests to `/login`, keeps authenticated users out of `/login`, and bounces role-mismatched navigation to the dashboard. Unknown paths redirect to the dashboard. All pages are lazy-loaded chunks. UI role checks are convenience only — every rule is enforced again by the API.

7.2 Pages

- **Login** — email/password card with error alert; on success routes to the dashboard.
- **Dashboard** — KPI cards, approval-queue alert, four charts, recent-requests table (§4.6).
- **Payment Requests** — server-paginated table with debounced filters (search, multi-status, department, date range).
- **Request form** — sectioned cards (Vendor & Invoice / Amount with live total / Classification / Documents) with client-side rules mirroring §6.4; Submit or Save-as-Draft.
- **Request detail** — header with status chip and context-aware action buttons (submit, edit, cancel, delete, approve, reject, schedule, mark paid), rejection banner, detail panels, documents with download, approval-chain timeline highlighting the current level, audit timeline, payment panel. Actions confirm through a shared dialog.
- **Approvals** — priority-ordered queue with inline Approve/Reject dialogs.
- **Payments** — three-tab queue (To Schedule / Scheduled / Paid) with Schedule and Mark-Paid dialogs.
- **Reports** — filter bar, summary cards per status, result table, Export-to-Excel button (opens the download with current filters).
- **Audit Trail** — admin log browser with text/action/date filters.
- **Users / Approval Levels** — admin CRUD dialogs as described in §4.9.

7.3 State management (Pinia)

Store	Holds
auth	Session user + <code>loaded</code> flag; getters <code>isAdmin/isApprover/isFinance/canApprove/canProcessPayments</code> ; actions <code>fetchUser</code> (memoised), <code>login</code> , <code>logout</code> (hard-redirects to <code>/login</code>).
meta	Dropdown data from <code>GET /api/meta</code> (categories, departments, currencies, payment methods, priorities, levels, upload limits), loaded once and force-refreshable.
notify	Global snackbar (text/color/visible) rendered bottom-right in the layout; <code>success()</code> / <code>error()</code> helpers.

7.4 HTTP & CSRF wiring

- One axios instance: `baseUrl '/api'` , `withCredentials` , `X-Requested-With` , `JSON Accept` .
- CSRF via the `XSRF-TOKEN` cookie (automatic) with the Blade `csrf-token` meta as first-load fallback.
- A response interceptor hard-redirects to `/login` on any 401 (outside the login page itself).
- `errorMessage()` flattens Laravel validation errors into a single readable snackbar message.
- File downloads (documents, Excel export) intentionally bypass axios via `window.open` — the session cookie carries auth.

7.5 Theme & visual language

- **Vuetify light theme:** primary `#256abf` , success `#008300` , warning `#eda100` , error `#d03b3b` , info `#2a78d6` , background `#f9f9f7` , surface `#fcfcfb` ; outlined comfortable inputs; navy nav drawer (`#10243e`).
- **Entity-stable status colors** — one hex per status everywhere (chips, charts, legends): paid `#008300` · scheduled `#2a78d6` · pending `#eda100` · approved `#1baf7a` · rejected `#e34948` · draft `#898781` · cancelled `#eb6834` . The donut's segment order is fixed and was validated for color-vision-deficiency separation (worst adjacent ΔE 21.2, deutan) — the order is a safety mechanism, not cosmetics.
- **Charts** (Chart.js): 2px surface gaps between donut segments, 4px rounded bar ends, recessive gridlines `#e1e0d9` , muted axis ink `#898781` , k-abbreviated money axes, tooltips everywhere; single-series bars carry no legend.
- Priorities: urgent `#d03b3b` · high `#ec835a` · normal `#52514e` · low `#898781` .

8 Configuration

Business lists in one PHP file; infrastructure in `.env`.

8.1 Business configuration — `config/paf.php`

Key	Values
<code>categories</code>	Goods / Inventory · Services · Utilities · Rent & Facilities · Marketing & Advertising · IT & Software · Logistics & Freight · Maintenance · Professional Fees · Travel & Accommodation · Other
<code>departments</code>	Finance · Procurement · IT · HR · Operations · Marketing · Sales · Logistics · Administration
<code>currencies</code>	AED · USD · EUR · GBP · SAR
<code>payment_methods</code>	bank_transfer → "Bank Transfer" · cheque → "Cheque" · cash → "Cash" · card → "Corporate Card"
<code>priorities</code>	low · normal · high · urgent
<code>max_documents</code> / <code>max_document_kb</code>	10 files / 10,240 KB (10 MB) each
<code>document_mimes</code>	pdf, jpg, jpeg, png, webp, doc, docx, xls, xlsx, csv, txt

Edits take effect immediately (validation and the `/api/meta` payload both read this file). Approval levels are *not* here — they live in the database and are managed from the admin UI.

8.2 Environment — `.env` highlights

Setting	Value / purpose
<code>APP_NAME</code> / <code>APP_ENV</code> / <code>APP_DEBUG</code>	<code>PAF</code> / <code>local</code> / <code>true</code> — set <code>APP_ENV=production</code> , <code>APP_DEBUG=false</code> before real use
<code>APP_URL</code>	<code>http://127.0.0.1:8084</code> — match the serve port (<code>php artisan serve --port=8084</code>)
<code>DB_*</code>	MySQL on <code>127.0.0.1:3306</code> , database <code>gallega_paf_db</code> , user <code>root</code> , empty password (local defaults — change in production)
<code>SESSION_DRIVER</code> / <code>CACHE_STORE</code> / <code>QUEUE_CONNECTION</code>	<code>all database</code> — no external services required
<code>SESSION_LIFETIME</code>	120 minutes
<code>FILESYSTEM_DISK</code>	<code>local</code> (private) — uploaded documents are never web-accessible directly
<code>MAIL_MAILER</code>	<code>log</code> — no outbound mail configured yet

9 Installation & Operations

From a clean checkout to a running platform.

9.1 Requirements

- PHP **8.3+** with extensions: pdo_mysql, mbstring, fileinfo, **zip**, **gd**, intl (zip/gd are needed by the Excel export)
- Composer 2.x · Node.js 20+ with npm · MySQL 8.x

9.2 First-time setup

```
# 1. Dependencies
composer install
npm install

# 2. Environment
copy .env.example .env          # then set DB_* to MySQL (database gallega_paf_db)
php artisan key:generate

# 3. Database (create schema first if needed)
# CREATE DATABASE gallega_paf_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
php artisan migrate
php artisan db:seed              # approval levels + accounts + demo data (optional but recommended)

# 4. Frontend
npm run build

# 5. Serve
php artisan serve --port=8084    # matches APP_URL; any port works if APP_URL agrees
```

Shortcut

The skeleton also ships `composer setup` (install → `.env` → key → migrate → npm install → build). Note it does **not** seed — run `php artisan db:seed` yourself afterwards.

9.3 Daily development

```
composer dev    # runs concurrently: artisan serve · queue:listen · pail (logs) · vite dev server
```

Or individually: `php artisan serve --port=8084` and `npm run dev` (Vite HMR). After frontend changes for production: `npm run build`.

9.4 Production checklist

- `APP_ENV=production`, `APP_DEBUG=false` (debug mode leaks stack traces in API errors), correct `APP_URL` (HTTPS).
- Dedicated MySQL user with a real password instead of `root` /empty.
- `php artisan config:cache route:cache view:cache` · `npm run build` · point the web server at `public/`.
- Serve over HTTPS and set `SESSION_SECURE_COOKIE=true`.

- Run a queue worker (`php artisan queue:work`) if mail/notifications are added later; configure a real `MAIL_MAILER` .
- Back up the database *and* `storage/app` (uploaded documents live there).
- Replace the seeded demo accounts/passwords before exposure beyond the dev machine (\$10).

10 Seeded Accounts & Demo Data

Everything `php artisan db:seed` creates.

10.1 Accounts

All seeded accounts share the password `password`.

Name	Email	Role	Level	Department	Job title
System Admin	<code>admin@paf.local</code>	admin	—	IT	System Administrator
Rachel Cruz	<code>requester@paf.local</code>	requester	—	Procurement	Procurement Officer
Daniel Reyes	<code>requester2@paf.local</code>	requester	—	Operations	Operations Coordinator
Liam Fernandez	<code>approver1@paf.local</code>	approver	1	Operations	Department Manager
Fatima Hassan	<code>approver2@paf.local</code>	approver	2	Finance	Finance Director
Omar Khalid	<code>approver3@paf.local</code>	approver	3	Finance	Chief Financial Officer
Grace Tan	<code>finance@paf.local</code>	finance	—	Finance	AP Accountant

10.2 Approval levels

Level	Name	Applies when total ≥
1	Department Manager	0 (every request)
2	Finance Director	10,000
3	CFO	50,000

10.3 Demo portfolio

59 realistic invoices spread over the past ~6 months across 12 vendors: 5 drafts, 8 pending approval, 6 approved, 4 scheduled, 28 paid, 5 rejected, 3 cancelled. Each carries a coherent lifecycle — chronologically consistent submissions, level-by-level approvals with comments, scheduling and payment references — and a matching backdated audit trail, so dashboards, queues, reports and timelines are meaningful immediately. Amounts are drawn from three bands aligned with the approval thresholds (≈200–9.5k / 10.5k–48k / 52k–220k AED, all with 5% VAT). The demo seeder is guarded: it does nothing if any invoice already exists.

11 Security Notes & Known Limitations

An honest register of design decisions and edges to be aware of.

11.1 Security posture

- Same-origin session auth with CSRF on every mutating call; login throttled (10/min); session fixation protection; deactivated accounts blocked at login.
- Server-side authorization on every endpoint: role middleware for finance/admin areas, per-record visibility checks for requests and documents, level-matching for approval actions. UI checks are cosmetic duplicates.
- Documents live on the private disk and stream only through the permission-checked download endpoint.
- Passwords bcrypt-hashed (12 rounds); audit rows are write-once (no update path, no `updated_at`).
- Approval history is snapshotted per request (level names copied), so renaming/removing levels never rewrites history.

11.2 Known limitations & deliberate trade-offs

Item	Detail & mitigation
Resubmission clears chain rows	Submitting a rejected request deletes its previous approval rows; the prior cycle survives only in the audit trail. Acceptable for an internal tool; revisit if formal approval history tables are needed.
No separation of duties in payments	Any finance/admin user can schedule and pay any approved request, including one they submitted. Add an ownership guard if policy requires it.
Create-then-submit is two transactions	<code>POST /api/invoices</code> with <code>action=submit</code> commits the draft first; if chain building fails (e.g. no active levels) the client gets 422 but the draft exists. Harmless — the draft is visible and resubmittable.
Reference-number race	<code>nextReferenceNo()</code> reads the last reference without locking; simultaneous creates could collide, in which case the unique index rejects one insert. Fine at internal volumes.
Draft-deletion audit rows unlinked	The <code>deleted</code> audit entry stores the reference in text but no <code>invoice_id</code> .
Admin approvals log an empty level	Approve/reject audit text interpolates the <i>actor's</i> level; admins (level-less) produce "Level approved...". Cosmetic.
Document downloads not audit-logged	Uploads/deletions are logged; reads are not. Add logging in <code>DocumentController@download</code> if read-auditing is required.
Unbounded <code>per_page</code>	List endpoints accept any page size. Cap it if the dataset grows large.
Visibility rule in three places	<code>Invoice::visibleTo()</code> , <code>InvoiceController::authorizeView()</code> , <code>DocumentController::authorizeAccess()</code> implement the same rule — keep them in sync when changing visibility policy.
Approver visibility is historical	An approver sees any request that has a chain row at their level, even after acting or when the request sits at another level. Intentional (context), but worth knowing.
Cancelled requests keep pending rows	Cancelling mid-chain leaves untouched pending approval rows attached to a terminal request. No functional impact.
Meta exposes inactive levels	<code>/api/meta</code> returns all levels including inactive ones (the admin UI needs them); consumers should filter on <code>is_active</code> where relevant.

Item	Detail & mitigation
Skeleton leftovers	<code>tailwindcss</code> deps and <code>database/database.sqlite</code> remain from the Laravel skeleton but are unused (live DB is MySQL).
No email notifications	Approvers learn of pending items from the dashboard alert/queue. Mail hooks are a natural next step (<code>MAIL_MAILER</code> currently <code>log</code>).

A Appendix

Reference tables.

A.1 Audit action catalog

Action	Written when	Extras
login / logout	Sign-in / sign-out	
created	Request created (draft)	linked to request
updated	Editable request modified	old/new value snapshots
submitted	Entered the approval chain	lists the required levels
approved	A level approved (advance or final)	actor + level in text
rejected	A level rejected	reason stored on request
cancelled	Withdrawn by owner/admin	
scheduled	Payment date set	date in text
paid	Payment executed	reference in text
deleted	Draft deleted	reference in text only
document_uploaded / document_deleted	Attachment added / removed	filename in text
report_exported	Excel export generated	
user_created / user_updated	Admin user management	role/level/dept/active snapshots on update
approval_level_created / _updated / _deleted	Admin level management	name/threshold/active snapshots on update

A.2 Status & priority color reference

Entity	Value → color
Status	draft #898781 pending #eda100 approved #1baf7a scheduled #2a78d6 paid #008300 rejected #e34948 cancelled #eb6834
Priority	urgent #d03b3b high #ec835a normal #52514e low #898781